Algoritmalar ve Veri Yapıları Bootcamp Ödev 4.
Çözümler

1. En uygun çözümün madeni paralarını bulmak için, problemin “recurrence” denkleminin uygulanması sırasında hangi madeni para birimlerinin minimum değerleri ürettiğini görmek üzere hesaplamaları geriye doğru izlememiz gerekir. Aşağıda verilen algoritma, iki ek dizi kullanarak bu geriye doğru izlemeyi gerçekleştirir. Coins adlı döndürülen dizinin sıfır olmayan üyeleri (başlangıçta tümü sıfır olarak ayarlanmıştır), çözümü oluşturan sonuçtaki madeni para değerlerini temsil edecektir.

```
Algorithm ChangeMaking(D[1..m], n)
    // Declare the array that will hold the result of the algorithm.
    Coins[1..m] ← 0       // Initialize all members to zero.
     // An array to hold history information about returns.
    History[1..n]
    // Initialize the base case for the problem's recurrence relation
    F[0] <-- 0
    for i ← 1 to n do
        temp <-- ∞              // or temp ← MAXINT
        j ← 1
        keep ← 0
        // for n ≥ j find the minimum among F[n-D[j]].
        while j ≤ m and i ≥ D[j] do
            // Below if statement is for the explicit implementation of
            // temp ← min(F[i – D[j]], temp)
            min←F[i-D[j]]
            if min < temp
                temp ← min
                // save the coin value, we will need it later.
                keep ← D[j]
            j ← j + 1
        F[i] ← temp + 1
        // Update the history with appropriate value.
        History[i] ← keep
i ← i -1
// Now let's collect coins back in history.
j ← 0
while i > 0 do
        Coins[j] ← History[i]
        i ← i – History[i]
```

```
            j ← j + 1
        return Coins        //Non-zero array members indicate the coins that make up the solution.
```

Yukarıdaki algoritma, tıpkı gerekli para üstünü tamamlamak için sadece madeni para adedini döndüren orijinal para üstü verme algoritmasında olduğu gibi, tam olarak n kez tekrarlanan bir dış döngü ve en fazla m kez tekrarlanan bir iç while döngüsü içerir Ayrıca, sonucu tutmak ve geçmişi saklamak için sırasıyla n ve m boyutlu iki ek dizi de tutar.

Bu nedenle, algoritmanın zaman karmaşıklığı, orijinal problemde olduğu gibi ***O(nm***) ve alan karmaşıklığı ***Θ(n+m)***'dir.

2. Dinamik programlama ile problemi çözmek için, daha küçük problemlerin çözümlerini birleştirerek çözebileceğimiz bir yineleme denklemi bulmamız gerekiyor. Daha küçük boyutlu problemler için:

1 <= i <= n tam sayı kümesinin değerleri a_1, a_2, a_3, ...an olsun.

1 <= j <= B, kontrol etmek istediğimiz toplam ağırlık olsun.

F(i, j), i tam sayı kümesi ve j toplamı için problemin optimum çözümünü (evet veya hayır, doğru/yanlış olarak) temsil etsin. İlk i tam sayının tüm olası alt kümelerini iki gruba ayırabiliriz:

- i. (son) tam sayıyı içeren alt kümeler ve
- i. (son) tam sayıyı içermeyen alt kümeler.

Bu gözlemden yola çıkarak:
Eğer i. tam sayıyı içermeyen alt kümeler arasında bir çözüm varsa, F(i-1, j) evet (veya doğru) olmalıdır. Sonuç olarak F(i, j) de evet (veya doğru) olmalıdır. Çünkü i. tam sayıyı eklemek bu çözüm üzerinde olumsuz bir etki yaratmayacak ve aynı çözüm i. tam sayıyı içeren durum için de geçerli olacaktır.
Eğer i. tam sayıyı içermeyen alt kümeler arasında bir çözüm yoksa, yani F(i-1, j) hayır (veya yanlış) ise, i. tam sayıyı ekleyerek ve i. tam sayıyı içeren alt kümeleri dikkate alarak bir çözüm elde edilebilir; öte yandan, i. tam sayıyı içeren alt kümeleri dikkate almak bile bir çözüme ulaşmak için yeterli olmayabilir. Dikkate alınması gereken üç durum vardır:

i. tam sayının değeri j'ye eşittir: O zaman i. tam sayının kendisi bile problemin koşulunu sağlar; Bu nedenle, F(i, j) evet (veya doğru) olmalıdır.

i. tam sayının değeri j'den büyükse: O halde, i. tam sayıyı dikkate alarak problemin koşulunu sağlama olasılığı yoktur. Zaten i. tam sayıyı dikkate almadan bir çözüm yoktu. Bu nedenle, F(i, j) hayır (veya yanlış) olmalıdır.

i. tam sayının değeri j'den küçükse: Problemin koşulunu sağlama olasılığının olup olmadığı, F(i-1, j-a_i) durumu için bir çözüm olup olmadığına, yani F(i-1, j-a_i) alt problemi için bir çözüm olup olmadığına (i. tam sayıyı içermeyen tam sayılar kümesinin toplam değeri j-a_i olan bir alt kümesinin olup olmadığı sorusu) bağlıdır. Eğer F(i-1, j-a_i) problemi için bir çözüm olsaydı, i.nci tam sayıyı eklemek toplamı j yapacaktı; dolayısıyla bunu daha küçük probleme eklemek, toplam değeri j olan bir alt kümenin var olma olasılığını mümkün kılacaktı.

Bundan dolayı, ilk i tam sayının olası alt kümelerinin toplam değeri j olan bir alt kümesi olup olmadığı sorusunun cevabı şöyledir:

F(i-1, j) veya F(i-1, j-a_i)

Bu gözlemler bizi aşağıdaki "tekrarlama" ilişkisine götürür.

F(i, j) = F(i-1, j) veya F(i-1, j-a_i) eğer a_i = j ise
F(i, j) = F(i-1, j) eğer $a_i \neq$ j ise

Dır. Aşağıdaki temel durumlarla (başlangıç şartları):

F(0, j) = false Boş bir kümenin, toplam değeri sıfırdan farklı pozitif bir tamsayı olan hiçbir alt kümesi olamaz.

F(i, 0) = true Herhangi bir kümenin (boş küme dahil), toplam değeri 0 olan bir alt kümesi (boş küme) vardır.

Algorithm *subsetWithTotalWeightExists*(A[1..n], B)
// **Input**: An array of integers (***a_1, a_2, …., a_n***,) and a positive integer B.
// **Output**: whether or not there is a subset of the given numbers whose total
// value is B. Returns true or false.

// Declare a 2-dimensional (nxB) array for the table
F[1..n, 1..B]
// Initialize the table with base case values.

```
// Initialize the table with base case values.
for i ← 0 to n do
        F[i, 0] ← ture
for j ← 1 to B do
        F[0, j] ← false

// From here is the realization of the recurrence relation
for i ←  1 to n do
        for j ← 1 to B do
                if  j = A[i] then
                        F[i,j] ← F[i-1, j]
                else
                        F[i, j] ←  F[i-1, j] or  F[i-1, j-A[i]]

return F[n, B]
```

Problemin boyutu iki değişkenle tanımlanır.: n and B

Temel İşlem: En içteki for döngüsünün içindeki karşılaştırma.

Zaman Karmaşıklığı: ***O(nB)***

Uzay Karmaşıklığı: ***O(nB)***